CP341 Final Paper

Walt Jones
Colorado College

Dan Schmidt
Colorado College

Stuart Sessions
Colorado College

October 16, 2024

Abstract

The underlying infrastructure for computer networks is seldom thought of in the modern era where applications that exist on top of the network are all the craze. Understanding how the physical hardware that underpins these networks is composed to host applications is a task that predates much of the literature available on the internet. In this paper, we set out to build a 5-layer network from scratch using C, Raspberry Pi 3B+, and custom-built Network Interface Cards (NICs). Using fiber cables connected to GPIO ports we manage to transmit information pertinent to our applications between Pis. Through this, we understand the general purposes each layer must serve, how to best implement them, and the drawbacks of our hardware.

1 Introduction

Modern computer networks are built upon years of development, which have made them extraordinarily reliable, and have pushed any logic that most developers might use to application programming, providing complete and robust functionality to accomplish the actual transmission of data. Beyond developers, computer scientists, and enthusiasts, the average user of our modern "internet" may have no understanding of a network's inner workings other than Google searches and basic domains. However, that was not always the case. Computer networks have only existed for 50 years, following the development of ARPANET in 1969 [6]. The advancement of network protocols like IPv4, TCP, ICMP, and UDP is a relatively recent phenomenon[10]. Today, packet-based networks have become a standard, formatting data and metadata into known packages to be decoded on either end of the wire. We chose to continue in this tradition and to use a 5-layer network. Those layers are the physical layer, which is the fiber the bits are sent on, the link layer, which establishes the connection and guarantees the ability to send bits on the wire; the network layer, which builds packets and manages addressing, forwarding, and route traffic; the transport layer, which encodes data with the Hamming 8-4 encoding algorithm, which we will discuss in depth later in this paper, and finally the application layer, which takes human applications and places them on the network by allowing for specific interpretation of the transmitted data. The goal of this project is to build each of these layers using optical cables, Rasberry Pi's, and custom NICs (Network Interface Cards) in the language C. C was chosen for its direct memory access features, its common usage in embedded hardware, and for performance reasons. The NICs have four IO ports, which accept optical cables. The signaling is done by pulsing light across the cables using the PiGPIO library.

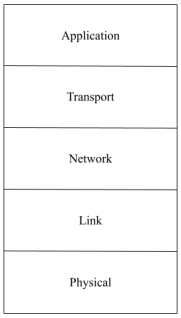


Figure 1: 5-Layer Network Model

2 Related Work

In previous iterations of the class in which this project was conducted, students managed to implement solutions to this problem. One group's paper, *Using Lasers to Send Cats* [2], was influential at the onset of our project. Their approach, using a 5-layer paradigm with routing tables and dynamic updates, aligned with what we hoped to create, and we hoped to improve upon their transmission rates and their transport layer by adding Hamming encoding to catch flipped bits. We also read *CP341 Final Paper* [8] to see how other groups approached this problem. We opted against ring topology for this project and followed their first router-client implementation.

3 Implementation

3.1 Link Layer

The link layer is the first layer of code to sit on top of the physical layer. The goal of the link layer is to send bits across the line and have the pure bits received and read into memory on either end. There are three main

issues that need to be solved to do this. These are [9]:

1. Framing
2. Error Detection and Correction
3. Clock Timing

Framing refers to how we sequence the bits with a start and stop sequence, or how the devices know when bits are done being read. Error detection (and correction) is how we can understand if a message was corrupted in transmission. Syncing clock timing guarantees that the clocks on the sending and receiving devices are *close enough* to send bits and read bits. We solve each of these issues in our link layer.

3.1.1 Framing

Implementing Manchester encoding effectively gives us our required start bit sequence of 1, then 0. We use these two values to track our read rate, which is the speed at which we check transitions. We reset the read rate for every message we read to ensure the least amount of clock drift. At the end of a message, we sent 8 1's, or *0xFF* and a zero to ensure our reader is low before the next read. This tells the reader the end of a sequence and lets it put those bits into memory. We need both the start and stop sequences to be correct to guarantee that bits end up in memory. One major bug we have not solved in the link layer is dealing with packets without a stop sequence. This is a case where it will leave our client "dead in the water," and a client reboot will need to occur.

3.1.2 Error Detection and Correction

We have pushed error detection and correction using Hamming encoding to the transport layer, where we encoded and decoded packets using 8-4 encoding. See the Transport Layer section for details.

3.1.3 Clock Timing

We use Manchester encoding [5] to solve the issue of clock timing. In Manchester encoding, each bit is represented by a transition: a low-to-high transition represents a 1, and a high-to-low transition represents a 0. This guarantees that there is a signal change in the middle of every bit period, allowing both data and clock information to be embedded in the signal. This embedded clock helps keep the sender and receiver synchronized without needing separate clock signals. Manchester encoding is part of a group of encodings called *bi-phase encoding* [4]. Another option we considered but opted against Non-Return-to-Zero (NRZ) methods represent 1 and 0 by on-off light levels (e.g., high for 1, low for 0), but without guaranteed transitions for every bit. As a result, long sequences of the same bit (e.g., a long sequence of 0s or 1s) can cause clock synchronization issues, since there are no signal changes during these periods. Manchester encoding avoids this by ensuring a transition in each bit period, even if the bit value remains constant. However, this comes at the cost of doubling the amount of time required compared to NRZ [3].

3.2 Network Layer

The network layer is responsible for building packets with data, distinguishing between control and application packets, routing data to the correct destination, and building a dynamic network map. In this project, we implemented two distinct network layers. One is a static client-router architecture, where up to four devices can be connected via one router. In this architecture, we statically define every device's address in a routing table that lives on a central router. See figure 2. Both architectures utilize the same packet structure. The first two bytes indicate the size of the data packet, the next two are the device address and the receiving address, and the remainder of the packet is data bytes, which the application layer handles, or the routing logic handles in the case of routing packets. See figure 3.

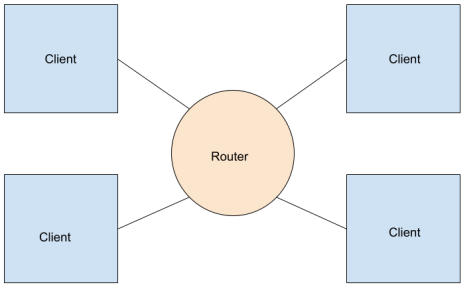


Figure 2: Client-Router Network

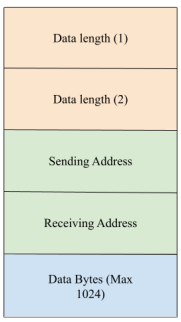


Figure 3: Packet Structure

This architecture is simple and easy to implement but lacks any logic for device disconnection, device routing changes, or any changes to addressing at run-time. Additionally, it can only support a maximum of four devices as the router only has four connection ports.

We also built a dynamic routing implementation using Distance Vector Protocol along with a simple Bellman-Ford Algorithm for shortest path detection. This is a significant shift from static routing, as it requires distinguishing between control packets and application packets, as well as implementing routing convergence, where every client sends its routing table to its neighbors, which propagates through the network. This theoretically allows for an arbitrary number of devices to be on the network, depending on resources available for converging the routing tables. This table contains information about the distance to every known device in the network. When a neighbor receives this table, it compares the distances with its own routing table to see if there's a shorter path to any destination. It uses the Bellman-Ford algorithm for this, a common shortest distance algorithm similar to Dijkstra's algorithm. In the main thread, when devices spin up, they announce their presence on the network by sending their empty routing table. All neighbors then reply with their own routing tables. When a device receives a routing update from a neighbor, it processes the information. For each destination listed in the update:

Algorithm 1 Routing Table Update Algorithm

```
Input: routing table from a neighbor, current routing table
for each destination in the received routing table do
    if destination is the device itself then
        skip this entry
    else
        add 1 to the distance metric
        if new route has a lower metric or neighbor is already the next hop then
            update the routing table with new next hop and metric
        end if
    end if
end for
if routing table changes then
    send the updated routing table to neighbors
end if
```

The devices keep track of when each route was last updated. If a route hasn't been updated in a certain amount of time, it is removed from the routing table. This ensures that routes to disconnected or unreachable devices are removed. This ideally works in a way where if a device is disconnected, the algorithm will recalculate the closest path to a node if it has more connections. This dynamic routing table takes substantial time to propagate across the network, in the worst case it could take *n-1* rounds of traffic for the full table to propagate, if each device only sends its table to it's neighbor. On a slow network, this takes a long time. [7] Combined with the unreliability of the link layer, this is a challenging task. See figure 4.

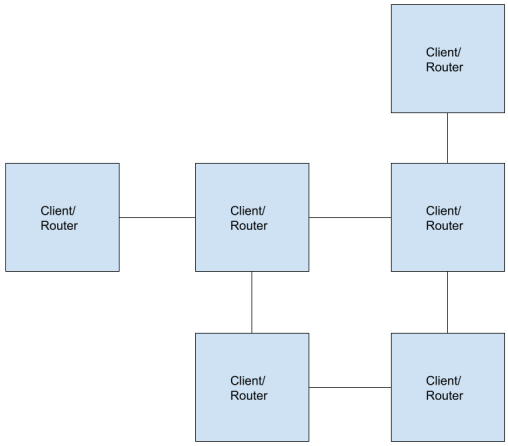


Figure 4: One *Potential* Configuration of Distance Vector

Although substantial work and many hours were put into testing this approach, we built our applications on top of the more limited, basic client-router architecture. Our network speed (capped around 50 bits per second) combined with a lack of logic for re-transmitting broken packets meant that the routing table could easily be broken. On top of that, issues with the reliability of transmission meant that the guarantees we would need to reliably build an N-sized network did not exist at the time of writing. Our network, therefore, is limited to 4 devices and one router, with the routing table being held on the router. When devices send a control packet to the router, it builds a new entry in the routing table. Then, all other devices can relay messages to it. Our network layer is initialized by sending a control packet to the reserved address *0x00*. When the router receives the update, it adds a mapping between the device's ID and the GPIO port it was received on. After this initial control message is sent, other devices can send packets to the device. There is currently no additional timeout logic in place.

3.3 Transport Layer

Within our network stack, the transport layer is responsible for ensuring reliable data transfer between devices over the network. While in our implementation not much other than error detection and correction occurs, in future implementations we would like this layer to manage flow control (ensuring the sender and receiver are not overwhelming/overwhelmed with messages) and payload segmentation (in the case that the application layer returns a payload that is large and would be more efficiently sent and received over multiple packets).

3.3.1 Hamming Encoding & Decoding

Due to the suboptimal electrical engineering in the design of our network interface cards (NICs), transmission errors such as bit flips are inevitable. It is essential to implement robust error detection and correction mechanisms to maintain the integrity of data being transmitted across the network.

We opted for the Hamming (8,4) code, to address

this issue. This encoding scheme works by combining four data bits with four parity bits into a byte. The parity bits allow theoretically for the detection of up to three bit errors per byte and the correction of up to two (in best case scenario, this is not guaranteed).

In our implementation, only the payload data is encoded. The decision to encode only the payload and not the packet headers or tails was made to enhance efficiency. By leaving the headers unencoded, routers can quickly process and route packets without needing to decode them, improving overall network performance. However, this does mean that headers are more susceptible to errors than payloads, though their concise size and importance make error detection less critical compared to the payload.

Hamming (8,4) encoding effectively allows up to 50% of the bits in a byte to be flipped and subsequently corrected upon reception. Although this doubles the size of the payload, the trade-off is acceptable given the improved resilience of the transmitted data, especially under error-prone conditions. This choice reflects a balance between redundancy, bandwidth efficiency, and error resilience.

Upon decoding, our implementation recalculates the parity bits based on the data bits, and compares the calculated parity to the received parity to detect alterations. The discrepancies between expected and received bits form the "syndrome", which indicates the position of an error. If the syndromes are non-zero and point to a valid bit position (1-8), the algorithm corrects the error/s by flipping the bits at the position/s indicated.

3.4 Application Layer

3.4.1 Design

The application layer is designed to inherent methods from the network layer. In a main function, the user selects the application and available addresses. Then, the application is launched based on the user input. The application layer has control over the data portion of our packets, and each application uses this differently. Our packets have a theoretical maximum size of $\tilde{1}6{,}000$ bytes based on our two byte data size header, but practically our applications use 50 byte packets for sending data. This is because the smaller the packet, the less time it will use the port allowing for other traffic to be sent across the network.

3.4.2 Messaging Application

The messaging app allows a user on one machine to send messages to a user on another machine. Upon initialization, the user is prompted to select another user on the network with whom they would like to chat with (the user on the other machine must also go through this process). From here users can send ASCII messages up to 140 characters long with one another. Due to its simplicity and quick runtimes, we used this application for tests throughout development and in our experimentation phase.

The app uses C's built-in ASCII encoding scheme. It takes each character and stores it as its corresponding byte representation (e.g., 'a' = '0x61'). These bytes are lumped together in a byte array which represents the entire message which is then passed on for Hamming encoding and then transmission. Upon decoding, this process is done in reverse. Thus, if the message was transmitted decently well, and Hamming decoding was successful, it should read the same as what was sent.sw In addition to the encoding and decoding functionality, this application prompts the user to select an address they would like to send to before sending. The router provides each of these options based on what other devices are on the network.

3.4.3 File Transfer Application

The file transfer application allows for small files (the smaller the less likely to have bit flips) to be transmitted between devices. It does this by breaking a file into n-packets, where each packet is ¡ 50 bytes large. We wrote a series of helper functions to accomplish this. The data is then divided into two metadata bytes per packet: the number of total packets being sent, and the current packet's number. This lets us give an error if we don't receive all the packets. That, combined with the rest of the data, is Hamming encoded and sent to the network layer. When the packets are received, the file is reassembled, Hamming decoded, and

3.4.4 Pong Application Attempt

The Pong application is not viable to use on our network due to incomplete functionality. However, we spent a significant amount of time developing the app so it is relevant to mention.

The Pong application is designed to create a simple game of pong that can be played across a network. The application features an initialization screen that waits for user input to start. In order for the app to start, it also needs a signal from the other user to indicate that both players are ready.

Once the game is started, it functions as an online game. One player can control one paddle, while the other player controls the other paddle. The program relies on synchronization between the two screens, so every time player 1 moves their paddle, it sends data across the network to update the position of player 2's paddle.

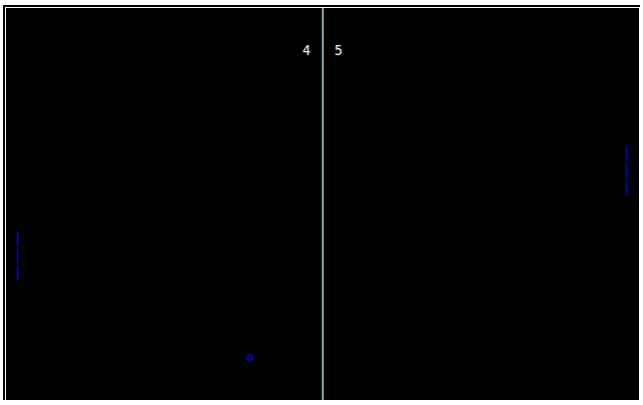


Figure 5: GUI Interface for Pong using ncurses[1]

Our implementation of Pong was based on a local version we found on GitHub, which we modified to be used over a network [1]. The modification we made was to remove the a/q control keys and instead read input data from a connected device to move the opposing paddle up/down. Additionally, every time input is captured with the arrow keys, the staging area for sending a message is updated and an output message is sent.or sending a message is updated and an output message is sent.

The Pong application has faced significant challenges due to the sheer frequency of data transmission between users. Quick message transmission is crucial for a seamless user experience. However, our network's low transmission rate results in significant delays in input and unresponsive interactions. Additionally, the large volume of data packets being sent and received between two clients increases the likelihood of errors, ultimately leading to frequent program crashes.

3.5 Challenges

3.5.1 Hardware

Throughout the course of this project, we experienced many setbacks and challenges. One substantial challenge was a lack of knowledge about hardware limitations. While we knew from prior classes that the hardware was prone to bit-flips, when we began to debug our network layer we noticed that some ports on the NIC devices are unreliable and have inconsistent default levels. As previously mentioned, a misconfigured initial reading value of a port messes up the Manchester encoding reading values, ruining the transmission. We managed this inconvenience by sending a packet ending in a zero to reset the reader value, and then restarting the program. Unfortunately, this did mean that often times when a bad packet was sent, we would end up having to repeat this process to restore our machine state.

3.5.2 Low Speed Networks

Our network can only transmit at speeds less than 75 bits per second, as defined in our sending logic. We dive deeper into actual speeds in our experimental results section. This causes issues with overwriting data before it has been fully read and processed, particularly with file transfer where we have to wait 2 seconds between sending packets to ensure we don't overwrite the previous packet. While there are more elegant solutions to this problem, we pushed that logic to the application layer.

3.5.3 Flow Control

The link layer's slow speeds mean that we have to be careful not to put more data on the wire before the data is read on the other side. We accomplish this currently by locking the reading/writing threads when in use, but this still leaves room for the send thread on one side to come de-synced from the read thread on the other and to send a packet while the read thread is still locked. This creates a situation where in the best case no packet is read, and in the worst case, an invalid packet is read and the program breaks.

4 Experimental Results

To test our implementation's speed and accuracy we wrote Python scripts that repeatedly sent messages using our messaging application at various BAUD rates. We were deliberate in trying to not let modifications to our codebase to accommodate experiments hamper the performance of essential functionality. However, we found inserting timers and passing in preset messages required altering the way our multi-threading worked substantially. Thus, if we had more time, we would refactor our experiments and codebase to produce results that we felt more comfortable with. Further, we were unable to do much experimentation with errors due to the variability in errors and not enough time to define each one.

4.1 Speed Performance

4.1.1 Slingshot

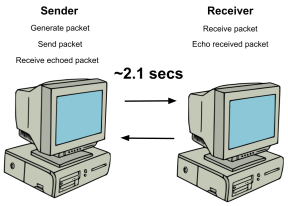


Figure 6: Slingshot Experiment Flow

Our first experiment measured the round-trip time for a 50-byte message, once packetized, to be sent and received at a BAUD rate of 100 to a receiver and then immediately returned to the sender. The timer was stopped once the sender received the message. Across 30 trials, the average time for this operation was approximately 2.1 seconds. While this speed might seem slow, we believe it to be near the upper bound of what's possible for a message of this size sent over the hardware we were provided.

During testing, we encountered bugs at the read stage, where the message often failed to be successfully decoded with Hamming when away. However,

we assume that the time difference between successful and unsuccessful decodes is negligible. Refactoring our multi threading logic to improve handling of this scenario would likely yield more accurate and meaningful results. It would also allow us to test variable BAUD rates to understand how much can be sent in a given period of time at a given rate.

5 Discussion & Future Work

While creating this project, we have learned that a network is only as good as its edge cases. There are many small things that could go wrong to throw off the network, from a user error to a hardware malfunction. The hidden differentiator for a network is how many things it can handle going wrong while maintaining its integrity.

Our network is designed to take in one packet, handle it for a designated application, and then begin to interpret the next input. While the pi's internal processing is fast, there are circumstances where the next message could be sent before the reading thread is ready for it, which can cause desynchronization between the reading function and the actual message being sent. Considering that the beginning of a packet contains important information about the structure of the packet, cutting off the first few bytes of a packet is problematic. In the future, we would address this issue by creating a message queue so that multiple successive messages could be handled. This would allow us to handle more inputs from either a user messaging very frequently, or an application that had to send rapid updates.

In real networks, incomplete and invalid packets will likely be received. Our current network doesn't handle these scenarios well, and is therefore particularly fragile. This problem is exacerbated by the NIC hardware we used for data transmission: inconsistencies in the hardware greatly increased the frequency of flipped bits and corrupted data. On occasion, a mistaken bit flip could occur during the stop sequence, causing problems with terminating a message read. In the future we would create link layer and network layer functionality to anticipate and smoothly handle these errors to prevent our network from crashing as frequently.

6 Conclusion

We have created a 5-layer network that can communicate information between Raspberry Pi's. We were given a physical layer of fiber optic cables that either send or omit light to signal data, and began by developing a link layer to send and receive pulses of energy. Once we were able to create this communication between two devices, we implemented the network layer by creating a router that appropriately routes network traffic to the expected user, preventing users from eavesdropping on private data. While not much needed to be done for our network's transport layer, we did add dynamic endpoint registration so the router can ensure reliable communication. Finally, we built two functioning applications that can be used on top of this network, a messaging app and a file transfer. These apps can be run by different devices across the network, and all traffic gets appropriately handled by the router. Through creating this network, we experienced firsthand many of the challenges that come with designing a network, such as synchronizing reading rates both at the link layer and network layer levels. Dealing with these issues and adjusting our network to mitigate errors has reinforced our understanding of network protocols, but also challenged our fundamental understanding of pointers, shared memory and other low-level programming fundamentals.

References

[1] Vicente Bolea. Pong-curses, github. URL: `https://github.com/vicentebolea/Pong-curses/blob/master/pong.c`.

[2] Jessica Hannebert Dylan Chapell and Tony Mastromarino. Using lasers to send cats: A custom networking implementation.

[3] Gorry Fairhurst. Non return to zero encoding, 2006. Accessed: 2024-10-14. URL: `https://erg.abdn.ac.uk/users/gorry/course/phy-pages/nrz.html`.

[4] Gorry Fairhurst. Manchester encoding, 2020. Accessed: 2024-10-14. URL: `https://erg.abdn.ac.uk/users/gorry/course/phy-pages/manc.html`.

[5] GeeksforGeeks. Manchester encoding in computer network. *GeeksforGeeks*, 2024. Last updated: July 16, 2024. Accessed: 2024-10-14. URL: `https://www.geeksforgeeks.org/manchester-encoding-in-computer-network/`.

[6] Michael Hauben and Ronda Hauben. Behind the net: The untold history of the arpanet and computer networking. *First Monday*, 3(8), 1998. URL: `https://firstmonday.org/ojs/index.php/fm/article/view/612/533`.

[7] Heikki Lindholm. The rip protocol, 1996. Accessed: 2024-10-15. URL: `http://www.netlab.tkk.fi/opetus/s38130/s96/papers/rip.pdf`.

[8] E. Moore E. Triplett M. Behenna, M. Hunter. Cp341 final paper, 2022.

[9] Shreyas Sharma. Data link layer. *Wevolver*, 2024. Last updated: 06 Jul, 2024. Accessed: 2024-10-14. URL: `https://www.wevolver.com/article/data-link-layer`.

[10] Andrew S. Tanenbaum and David J. Wetherall. *Computer Networks*. Pearson, Amsterdam, The Netherlands, 5th edition, 2011. URL: https://csc-knu.github.io/sys-prog/books/Andrew%20S.%20Tanenbaum%20-%20Computer.pdf.